

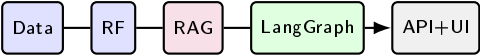
EduCoach AI

Hybrid ML + Multi-Agent Tutoring — Implementation Deep Dive

Mohamed Yassine BAHAJOU

Final Project — Hybrid ML & Multi-Agent Systems

August 5, 2026



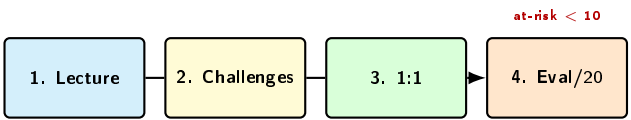
Implementation deep dive · 24×11 · `src/ml rag agents api ui`



Bonjour, et merci d’être là. Aujourd’hui je veux vous montrer EduCoach AI — pas seulement l’idée, mais comment on l’a vraiment construit.

C’est un outil de coaching pour une classe de vingt-quatre débutants, sur un programme Python de onze jours. Je vais suivre la bande en bas, de gauche à droite : data, machine learning, retrieval, agents, puis l’API et l’interface — et je finirai avec une démo live.

À la fin, vous saurez ce que fait chaque partie, et comment elles s’emboîtent.



- Cohort: 24 beginners · 11-day Python SAS
- Need: early at-risk + weak-point detection for same-day tutoring
- Impact: less dropout; less accumulated concept gap (*décalage*)

2026-08-05

Context — a SAS day

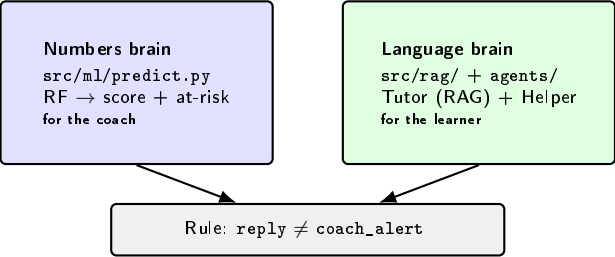
D’abord, le quotidien. Une journée a quatre moments : un cours, des challenges, un suivi one-to-one, et une évaluation du soir sur vingt.

Le problème, c’est l’échelle. Avec vingt-quatre débutants, un coach ne peut pas tout vérifier avant le test du soir. Quelqu’un décroche en silence, et on le voit seulement quand les notes arrivent — trop tard pour aider le même jour.

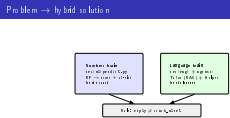
Donc il faut une détection précoce : signaler qui est at-risk, et sur quels topics, à temps pour aider le même jour. Et ça compte vraiment, parce que dans un programme intensif, un petit trou le jour trois grossit le jour quatre et cinq. Ce décalage, c’est ce qui fait abandonner les débutants.



Problem → hybrid solution



Problem → hybrid solution



L'idée forte de cette slide : on a construit *deux cerveaux*, pas un seul gros prompt qui fait tout. Le premier, c'est le numbers brain. Il regarde l'activité de l'apprenant — exercices attempted, solved, hints used — et un Random Forest en fait un predicted score et un flag at-risk pour le coach. Ça vit dans `src/ml/predict.py`. Le second, c'est le language brain. Il répond aux questions de l'apprenant — théorie ou code cassé — en allant chercher la bonne leçon, puis en laissant des agents spécialistes répondre. Ça, c'est `src/rag/` et `src/agents/`. Et une règle les lie : l'apprenant ne voit que `reply` ; le coach voit `coach_alert`. Le signal de risque ne touche jamais la réponse de l'apprenant. Gardez cette règle — elle revient à chaque couche.

Implementation map (phases → code)

Phase	Module	Key entrypoints
0 Data	generate_data.py	CSV panel 24×11
1 EDA	01_EDA_and_Data_Gen.ipynb	quality, distributions, correlations
2 ML	src/ml/predict.py	predict_student, .pkl
3 RAG	ingest.py/retrieval.py	Chroma educoach_lessons
4 Agents	orchestrator.py	chat() LangGraph
5 Product	api/ + ui/	FastAPI + Streamlit wrappers

Phase 5 does **not** reimplement ML/RAG/agents — it only exposes them.

Implementation map (phases → code)

Cette slide, c'est la carte du talk — chaque phase et où elle vit dans le code.

Ça commence par la data : generate_data.py écrit le dataset, et le notebook EDA l'explore — qualité, distributions, balance, et un check de leakage avant de modéliser.

Ensuite le machine learning : l'entraînement sauve random_forest.pkl, et predict_student le sert. Puis le retrieval : ingest construit le vector store, et retrieve_context nourrit le Tutor.

La phase quatre, c'est l'orchestration, et voici le point clé : l'API n'appelle jamais les agents directement — elle a un seul entry point, orchestrator.chat. Et la phase cinq, la couche produit, est volontairement fine — juste des wrappers autour de tout ce qu'il y a en dessous.

Donc on commence là où naissent les chiffres — le generator.

Phase	Module	Key entrypoints
0 Data	generate_data.py	CSV panel 24×11
1 EDA	01_EDA_and_Data_Gen.ipynb	quality, distributions, correlations
2 ML	src/ml/predict.py	predict_student, .pkl
3 RAG	ingest.py/retrieval.py	Chroma educoach_lessons
4 Agents	orchestrator.py	chat() LangGraph
5 Product	api/ + ui/	FastAPI + Streamlit wrappers

Phase 5 does **not** reimplement ML/RAG/agents — it only exposes them.

- File: `data/synthetic/generate_data.py` → `student_performance.csv`
- Nested loops: `students` × `days`; `RANDOM_SEED` for reproducibility
- Score: `previous` + `success rate` + `noise`; `penalty` if `low solves` + `high hints`; `clip` [0,20]
- Derive `at_risk=(score<10)`; chain `previous_eval_score` from yesterday

2026-08-05

EduCoach AI

└ Data — generator design

File: `data/synthetic/generate_data.py` → `student_performance.csv`
Nested loops: `students` × `days`; `RANDOM_SEED` for reproducibility
Score: `previous` + `success rate` + `noise`; `penalty` if `low solves` + `high hints`; `clip` [0,20]
Derive `at_risk=(score<10)`; chain `previous_eval_score` from yesterday

Pourquoi générer nos propres data ? La transparence. Nos hypothèses sur ce qui fait qu’un apprenant galère sont dans le code, pas cachées dans des logs sales — et tout le monde peut reproduire, sans souci de privacy.

Ce sont deux boucles imbriquées : vingt-quatre `students` sur onze `days`, donc deux cent soixante-quatre rows. Un `seed` fixe donne toujours les mêmes data.

Chaque row reçoit le `topic` du jour et une activité simulée — `attempts`, `solves`, `hints`, `time`. Le `score` est réaliste : `previous score` plus `success rate`, un peu de `noise`, une pénalité si peu de `solves` avec beaucoup de `hints`, puis `clip` entre zéro et vingt.

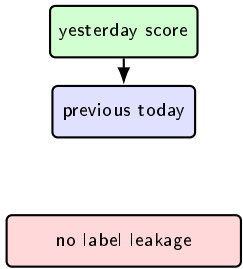
Deux détails comptent plus tard. `At-risk`, ça veut juste dire `score` en dessous de dix. Et le `previous-day score` n’est pas aléatoire — c’est littéralement le `score` d’hier pour le même apprenant. C’est ça qui en fait de vraies data jour après jour.

FEATURE_COLUMNS (in X)

- day, topic
- exercises__attempted / solved
- hints__used, time__spent__minutes
- previous__eval__score

Forbidden in X

- student_id, today_eval_score, at_risk



2026-08-05

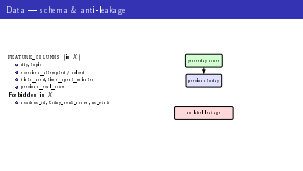
Data — schema & anti-leakage

Cette slide, c’est de la discipline. Le moyen le plus simple de gâcher un projet ML, c’est de donner au modèle une info qu’il n’aurait pas en vrai.

Notre contrat, c’est une constante, FEATURE_COLUMNS dans predict.py. Elle liste les sept inputs que le modèle peut voir : day, topic, exercises attempted et solved, hints, time, et le score d’hier. La même liste sert pour le training et la prediction, pour qu’elles ne divergent pas.

Remarquez ce qui n’y est *pas*. Le student id est dans les data et l’UI, mais jamais dans le modèle — on veut qu’il apprenne le comportement, pas qu’il mémorise les personnes.

Et la règle qu’on ne casse jamais : le score d’aujourd’hui, et le label at-risk qui en découle, sont des outputs — jamais des inputs. S’ils rentraient dedans, le modèle lirait la réponse, et les metrics auraient l’air super sans rien vouloir dire. Le risk board du coach suit la même règle et appelle le même predict_student.



- Keep previous + activity; One-Hot topic (nominal)
- Do not over-trust day (redundant with topic)
- Coach metric: **at-risk recall** (misses cost more)
- Interactions hints×solves ⇒ choose Random Forest

2026-08-05

EduCoach AI

EDA — what changed the build

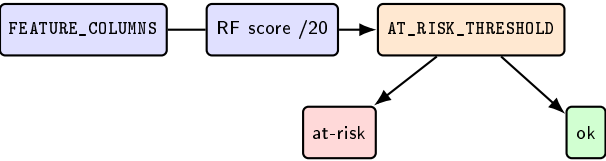
Les choix de modeling n'étaient pas au hasard — ils viennent directement de l'exploration des data. Voici ce qu'on a vu.

Quatre choses ressortent. Le score d'hier suit fortement celui d'aujourd'hui. Hints en hausse plus solves en baisse, ça veut dire *risque* en hausse — un vrai red flag. Topic est juste une catégorie sans ordre. Et day, seul, est un faible prédicteur, car chaque day correspond déjà à un topic.

Ça a donné trois décisions : one-hot encode sur topic plutôt qu'un faux ordre ; prioriser le recall at-risk plutôt que le dernier dixième de point ; et choisir un Random Forest, parce que le vrai signal, c'est l'*interaction* entre hints et solves — ce qu'un modèle linéaire rate.

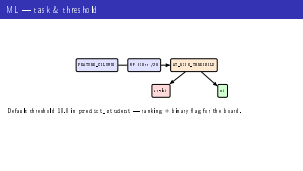
C'est exactement ce qui mène à l'implémentation ML suivante.

- [Keep previous + activity; One-Hot topic \(nominal\)](#)
- [Do not over-trust day \(redundant with topic\)](#)
- [Coach metric: at-risk recall \(misses cost more\)](#)
- [Interactions hints×solves ⇒ choose Random Forest](#)



Default threshold 10.0 in predict_student — ranking + binary flag for the board.

ML — task & threshold

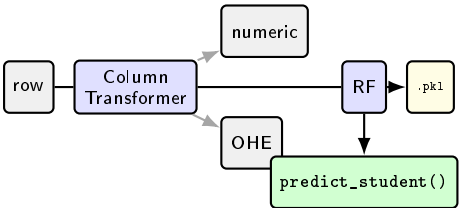


Voici un choix qui surprend : même si le coach veut un oui/non sur le risque, on prédit d’abord un score *continuous*, puis on convertit.

Pourquoi ? Parce qu’un chiffre sur vingt permet de ranker la classe. Savoir l’ordre de ceux qui galèrent, c’est bien plus utile qu’une liste plate de noms.

La conversion est dans `predict_student` : un threshold, dix par défaut, et at-risk veut dire score en dessous.

Et ce ranking, c’est exactement pourquoi on n’a pas juste fait un classifieur. Un predicted three et un predicted eight sont tous les deux at-risk, mais pas aussi urgents — le coach doit d’abord aller vers le three. Un simple oui/non jette ça à la poubelle.



- Artifacts: `models/random_forest.pkl`, `model_metrics.json`
- Same function used by Analyzer and POST `/predict_today`

2026-08-05

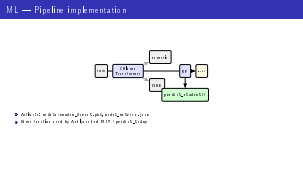
ML — Pipeline implementation

Voici comment le modèle est vraiment assemblé. On n’a pas juste donné les data brutes à un Random Forest — on a tout enveloppé dans un seul pipeline scikit-learn.

Dedans, un `ColumnTransformer` traite les deux types de features différemment : les numerics passent, topic passe en one-hot. Ça nourrit un `RandomForestRegressor` dont la depth, les leaves et le nombre de trees viennent d’un grid search, pas d’un guess. Preprocessing et modèle sont sauvés ensemble dans `random_forest.pkl`, avec un fichier `metrics` à côté.

Le sauver comme un seul objet, c’est ce qui rend le serving sûr. Au runtime, `predict_student` construit une frame d’une row dans le bon ordre de colonnes, prédit, applique le threshold, et renvoie le score, le flag, et le threshold. Comme le preprocessing est dans l’artifact, prediction et training ne peuvent pas diverger en silence.

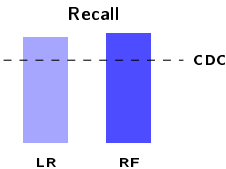
Un point de plus : l’Analyzer et l’endpoint `/predict_today` appellent la même fonction. Une seule source of truth — donc le dashboard et les agents ne peuvent jamais se contredire.



Metric	LR	RF
RMSE	2.73	2.65
R^2	0.80	0.81
Recall	0.96	1.00

Soft RMSE < 2.5 slightly

missed — recall-first.



2026-08-05

EduCoach AI

ML — metrics we ship

ML — metrics we ship

Metric	LR	RF
RMSE	2.73	2.65
R^2	0.80	0.81
Recall	0.96	1.00

Soft RMSE < 2.5 slightly



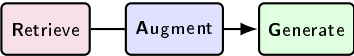
Maintenant les chiffres, honnêtement — y compris là où on a un peu raté. On compare toujours à un linear baseline, pour juger si le forest mérite sa complexité.

Sur le test split, le Random Forest a environ deux virgule six cinq d'erreur, un R-squared autour de zéro virgule huit un, et — le plus important — un recall at-risk de un point zéro, donc zéro struggler manqué. Il bat le baseline partout.

Un point honnête : la cible soft, c'était une erreur sous deux virgule cinq, et on est un peu au-dessus à deux virgule six cinq. On ne le cache pas. Le recall passe d'abord, parce que rater un apprenant qui galère coûte bien plus qu'un score un peu bruité — on préfère une fausse alerte à un miss.

Les importances nous rassurent aussi : solves et previous score dominant, day est faible — juste comme l'EDA l'avait prédit.

Voilà le numbers brain, pour le coach. Mais l'apprenant a encore besoin de réponses — depuis *nos* leçons, pas l'imagination du modèle. C'est le retrieval, juste après.



- Offline: `src/rag/ingest.py` → `vector_db/`
- Online: `retrieval.py` → Top-*K* context string
- Generate: `concept_tutor.py` + `get_llm()`

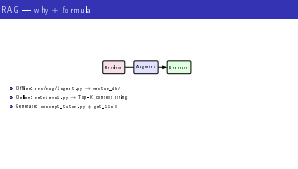
2026-08-05

└ RAG — why + formula

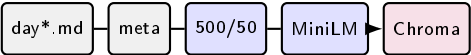
Voici le problème que le retrieval résout. Si on demande à un language model tout seul, il répond depuis son training général — fluide, mais il peut inventer en silence une syntaxe qui contredit ce qu'on a enseigné. Pour un outil pédagogique, non.

Le RAG règle ça en trois étapes qui forment le nom : *Retrieve* les bons chunks de leçon, *Augment* le prompt avec, puis *Generate* — donc le modèle répond avec nos leçons ouvertes devant lui.

Dans le code, ce sont deux horloges plus la génération. `ingest.py` prépare la knowledge une fois, offline. `retrieval.py` va chercher les chunks à chaque question, online. Et `concept_tutor.py` génère la réponse avec le modèle que renvoie `get_llm()`. Je prends les deux côtés l'un après l'autre.



RAG — offline ingest (implementation)



- Filename regex `dayNN_topic.md` → metadata
- `RecursiveCharacterTextSplitter`; collection `educoach_lessons`
- ~131 chunks; rebuild: `python -m src.rag.ingest -rebuild`

RAG — offline ingest (implementation)

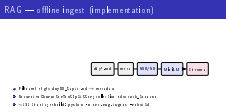
D'abord l'ingest offline — le pipeline qui transforme nos leçons Markdown en quelque chose de searchable. Il tourne une fois, quand les leçons changent.

Ça commence avec `load_lesson_documents` : il lit chaque fichier nommé `dayNN_topic`, tire `day` et `topic` du filename comme metadata, et saute le README.

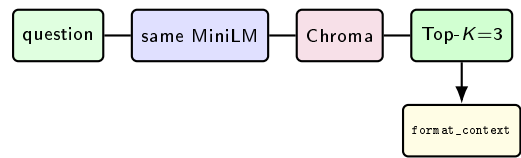
Puis le chunking. Les leçons sont trop longues pour embedder d'un coup, donc un splitter les coupe en morceaux d'environ cinq cents caractères, avec un overlap de cinquante. L'overlap compte — ça évite de couper un exemple de code en deux. Chaque chunk reçoit ses metadata et un id.

Ensuite chaque chunk est embeddé avec MiniLM sur CPU et stocké dans Chroma, collection `educoach_lessons` — environ cent trente chunks après enrichissement des leçons.

Une note pratique : un changement de version Chroma a cassé un ancien index, donc `-rebuild` l'efface et le recrée proprement. Petit détail, mais ça nous a sauvés.



RAG — online retrieval (implementation)



- `retrieve_context(question)` uses `RAG_TOP_K`
- Context labels Day/topic/source for citations
- Enriched lessons: code + step-by-step + Output

RAG — online retrieval (implementation)

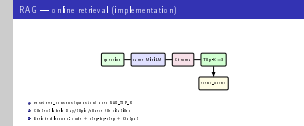
Maintenant le côté online, qui tourne à chaque question d'apprenant. La fonction, c'est `retrieve_context`, et elle fait l'inverse de l'ingest.

Elle embedde la question avec le *même* modèle MiniLM et cherche dans Chroma les chunks les plus proches — top trois par défaut, assez pour répondre sans noyer le prompt.

Puis `format_context` les dispose avec un petit header — day, topic, source — puis le texte. C'est ce que le Tutor lit et cite, d'où les réponses du type « ça vient du day four ».

Une règle non négociable : le modèle d'embedding au query time doit matcher celui de l'ingest. Sinon, les vectors vivent dans des espaces différents et la search renvoie du nonsense en silence — pas d'erreur, juste de mauvais résultats. Invisible, et dangereux.

Et une leçon apprise à la dure : au début le Tutor disait souvent « I'm not sure ». Le modèle allait bien — nos leçons étaient trop maigres. Une fois qu'on a ajouté de vrais exemples et l'output attendu, les réponses sont devenues solides. Le retrieval n'est jamais meilleur que la knowledge derrière.



- Tutor prompt: ONLY from context; cite Day X; include code/Output
- If context weak → say unsure (no invention)
- Update path: edit MD + re-ingest (not retrain an LLM)

	RAG (ours)	Fine-tune
Update	re-ingest	retrain
Citations	metadata	harder
Cost	light	heavy

RAG — grounding & vs fine-tuning

Récupérer les bons chunks, ce n'est que la moitié — il faut aussi forcer le modèle à rester dedans. C'est le system prompt du Tutor.

Il est strict : répondre seulement depuis le context donné, citer le day, inclure le code et l'output s'ils sont là, ne jamais inventer, et admettre quand le context ne suffit pas plutôt que bluffer. Cette honnêteté, c'est une feature. On nous demande souvent pourquoi on n'a pas juste fait du fine-tuning sur les leçons. La praticité. Le fine-tuning met les leçons dans les weights — lent, cher à mettre à jour, et faible sur les citations. Avec le RAG, mettre à jour est trivial : éditer le Markdown, re-ingest, terminé. Pour un curriculum qui bouge, cette flexibilité gagne.

Mais retrieval et un bon prompt, ce n'est encore qu'une capability, pas un produit. Quelque chose doit décider *quand* utiliser le Tutor et garder le tout safe. C'est l'orchestration — juste après.

- Tutor prompt: ONLY from context; cite Day X; include code/Output
- If context weak → say unsure (no invention)
- Update path: edit MD + re-ingest (not retrain an LLM)

	RAG (ours)	Fine-tune
Update	re-ingest	retrain
Citations	metadata	harder
Cost	light	heavy



- Tools here: `retrieve_context`, `predict_student`
- PoC: prompt + one tool inside LangGraph (not a huge ReAct loop)

2026-08-05

EduCoach AI

Agents — chatbot / tool / agent

Le mot « agent » est flou partout, donc je le fixe en trois niveaux.

Un chatbot, c’est juste un prompt qui parle — pas de tools, pas de spécialisation. Un tool, c’est une capability sans conversation ; les nôtres sont `retrieve_context` pour les leçons et `predict_student` pour le risque. Un agent est au-dessus : un *role* défini, éventuellement son propre modèle, plus l’accès à ces tools.

Et je serai honnête sur le style qu’on a bâti : structuré, pas un swarm libre. Chaque spécialiste est fin — un prompt ciblé plus un tool — dans un graph fixe. C’était volontaire : ça garde le système explicable et testable, ce que veulent un jury et un maintenir.



- Tool here: `retrieve_context`, `predict_student`
- PoC: prompt + one tool inside LangGraph (not a huge ReAct loop)

Agent	Speaks?	Tool	File
Supervisor	No	regex	orchestrator.py
Concept Tutor	Yes	RAG+LLM	concept_tutor.py
Code Helper	Yes	LLM hints	code_helper.py
Analyzer	No	RF	performance_analyzer.py

- Shared: llm.py (LLM_PROVIDER), state.py (AgentState)

2026-08-05

Cette table, c'est le casting — et la colonne clé, c'est qui parle à l'apprenant ou qui travaille en silence. L'orchestrator tient le graph, l'entrée publique chat, et le supervisor qui route. Le Concept Tutor parle, pour la théorie — retrieval plus une réponse grounded. Le Code Helper parle aussi, pour le debugging, mais seulement en hints. L'Analyzer ne dit jamais un mot à l'apprenant — il lance le Random Forest pour le coach.

En dessous, deux pièces partagées. llm.py est une petite factory qui renvoie Gemini, Ollama, Groq ou OpenAI. On tourne Gemini Flash-Lite — et il y a une histoire : on a d'abord essayé un modèle Ollama local et ça a freezé la machine, donc cette factory nous a laissé changer de provider sans toucher un seul agent. Et state.py définit AgentState, les data partagées — slide suivante.

Agents — files & contracts

Agent	Speaks?	Tool	File
Supervisor	No	regex	orchestrator.py
Concept Tutor	Yes	RAG+LLM	concept_tutor.py
Code Helper	Yes	LLM hints	code_helper.py
Analyzer	No	RF	performance_analyzer.py

► In llm.py: llm.py (LLM_PROVIDER) app (AgentState)

Agents — AgentState (shared bag)

```
student_id, message,  
route, history_text, reply,  
blocked, block_reason,  
features,  
predicted_score, at_risk,  
coach_alert
```

- Nodes return **partial** updates
- LangGraph merges into state
- Tutor sets **reply** only
- Analyzer never touches **reply**

EduCoach AI

2026-08-05

- Agents — AgentState (shared bag)

Agents — AgentState (shared bag)

```

random_id, range,
name, his_age, sex,
his_id, his_age, sex,
his_age,
predict_age, sex,
his_age

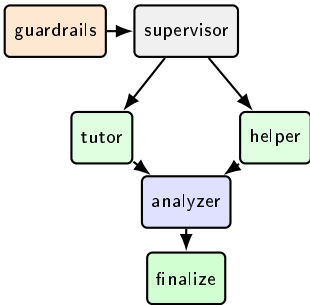
```

- Will often **partial** up to
- Ligands may be **kinetic**
- Transition complex
- As **kinetic** is a complex

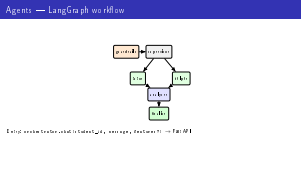
Si les agents sont le casting, `AgentState` est le script partagé qu'ils lisent et écrivent pendant qu'une request traverse le graph. Ça compte parce que c'est ainsi que la règle produit est appliquée.

Trois groupes de fields. Les inputs qui démarrent le run — student id, message, features optionnelles. Les fields de contrôle qui orientent — la route, si c'était blocked et pourquoi, l'history récente. Et les outputs — le reply de l'apprenant, le predicted score, le flag at-risk, et le coach alert séparé.

Le truc qui rend ça sûr : chaque node renvoie seulement une update *partial*, et LangGraph merge. Le Tutor renvoie juste un reply ; l'Analyzer renvoie seulement les fields de risque. Comme l'Analyzer ne renvoie jamais le field reply, c'est structurellement *impossible* que la logique de risque écrase ce que voit l'apprenant. La règle two-channel, ce n'est pas de bonnes intentions — c'est la forme des data.



Entry: `orchestrator.chat(student_id, message, features?)` → FastAPI



Ce diagramme, c'est le cœur du système, donc je trace une request dedans. C'est un state graph sur AgentState, avec six nodes : guardrails, supervisor, tutor, helper, analyzer, finalize.
Ça se lit de haut en bas. Chaque request passe d'abord par les guardrails, pour la safety. Puis le supervisor décide : Tutor, Helper, ou — si c'était blocked — directement la fin. Quel que soit le spécialiste qui répond, les deux chemins se rejoignent à l'Analyzer, puis finalize.
Pourquoi un graph et pas juste une suite d'appels de fonctions ? Parce que le graph rend le flow *explicit* et drawable — cette image, c'est littéralement le code — et ajouter un spécialiste, c'est juste un node et un edge.
Et de l'extérieur, tout se replie en une fonction : chat, avec student id, message, et features optionnelles. Elle lance le graph et renvoie du JSON propre. C'est la seule chose que l'API appelle jamais.

2026-08-05

Agents — routing, tutor, helper (impl.)

Agents — routing, tutor, helper (impl.)

```
• The sig: theory_hint → tutor; code_hint → helper; else tutor
• The "for loop" takes a list of (theory_hint, code_hint, debug_word)
• The: retrieve → format → System+history+context+Q → LLM
• Helper: same get_llm(), stricter prompt — never full solution
```

- **Routing:** THEORY_HINTS → tutor; CODE_HINTS → helper; else tutor
- Fixed “for loop” false-positive (theory-first unless debug words)
- **Tutor:** retrieve → format → System+history+context+Q → LLM
- **Helper:** same get_llm(), stricter prompt — never full solution

Deux décisions façonnent l’expérience de l’apprenant : comment le supervisor route, et comment chaque spécialiste répond.

Le supervisor utilise du keyword matching. Une phraséologie de concept — « what is », « explain » — va au Tutor ; des mots de code — error, bug, traceback — vont au Helper ; le flou va par défaut au Tutor. Choix pragmatique : rapide, gratuit, explicable, et facile à upgrader plus tard vers un model router.

Un petit bug instructif : « for » ressemble à du code, donc « What is a for loop ? » allait d’abord au Helper. Le fix, c’est le routing theory-first — rester au Tutor sauf s’il y a de vrais mots d’erreur. Le routing doit respecter l’intent, pas juste repérer des keywords.

Côté réponse : le path Tutor, c’est le flow retrieval — fetch, format, construire le prompt avec history et question, appeler le modèle. Le Helper utilise le même modèle mais des règles plus strictes : hints seulement, jamais la full solution, même si on la demande. Cette retenue, c’est le but — on veut qu’ils apprennent, pas qu’ils copient.

Analyzer + alerts

- no features → skip
- else predict_student
- at-risk → coach_alert
- API append_alert

Memory + safety

- last 5 turns (short_term.py)
- input_validator
- prompt_injection
- defense in depth

2026-08-05

EduCoach AI

Agents — analyzer, memory, guardrails

Agents — analyzer, memory, guardrails

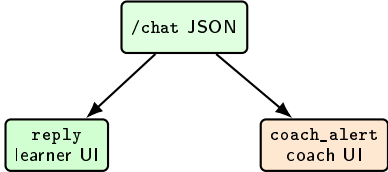
Trois pièces autour des spécialistes : l'analyzer, la memory, et la safety.

L'Analyzer, c'est le pont vers le coach. Pas de features ? Il ne fait rien. Features présentes ? Il appelle le même predict_student, et si l'apprenant est at-risk il écrit un coach-alert. L'endpoint chat sauve ça sur le dashboard coach — sans que l'apprenant le voie jamais.

La memory est légère exprès : les cinq derniers turns par apprenant. Les guardrails chargent cette history pour que le Tutor ait du context, et finalize réécrit le nouveau turn. Assez pour une vraie conversation — et oui, ça reset au restart. La memory persistante, c'est un souci production, pas PoC.

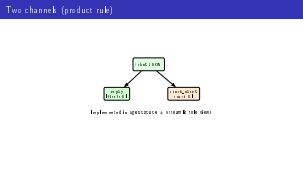
La safety tourne en premier. Un check attrape les inputs dangereux, un autre le prompt injection — « ignore your instructions », ou la demande de full solution. Si l'un des deux tire, on renvoie un reply sûr et on saute les spécialistes, donc rien de suspect n'atteint le modèle. Defense in depth — plusieurs couches, pas un seul mur.

Two channels (product rule)



Implemented in AgentState + Streamlit role views

Two channels (product rule)

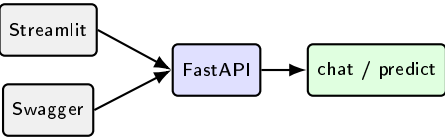


Je reviens souvent sur cette règle parce que c’est le cœur éthique et produit — et elle est appliquée partout, pas seulement racontée.

Elle court de bout en bout : deux fields séparés dans le state, deux fields dans le JSON de l’API, deux vues dans l’interface. La réponse de l’apprenant et l’alert du coach ne sont jamais le même objet.

Concrètement, la bulle de l’apprenant est liée seulement à reply ; la liste d’alerts du coach est liée à coach-alert via son propre endpoint. L’apprenant n’a tout simplement aucun canal pour recevoir le signal de risque.

Pourquoi tant tenir ? Parce que dire à un débutant « you’re at risk » dans son chat, c’est décourageant et ça peut devenir self-fulfilling. C’est le coach qui doit agir, doucement et en personne. Avec ça posé, je montre la couche produit fine — la couche HTTP.



POST /chat · POST /predict_today · GET /coach/risk_board · GET /coach/alerts
Pydantic schemas validate; routes are thin wrappers over existing Python

2026-08-05

API — why + routes

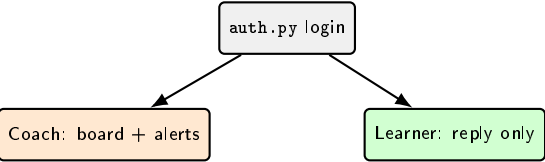


La couche API est volontairement *boring* — et c’est un compliment. Elle n’ajoute aucune intelligence ; elle expose juste les cerveaux qu’on a construits via du HTTP propre et validé.

Chaque request et response a un schema Pydantic, donc le contrat est documenté et validé — les mauvaises data sont rejetées tôt avec une erreur claire, pas un crash au fond.

Les routes sont de fins wrappers. Chat appelle `orchestrator.chat`, sauve un coach alert s’il y en a un, et renvoie le reply plus route et score. Predict est essentiellement un alias de `predict_student`. Et le risk board boucle sur les vingt-quatre learners et trie les plus at-risk en premier — encore une fois, activity features seulement, pas de leakage.

Un bonus sympa : un produit API-first donne une page Swagger gratuitement. On l’a utilisée pour smoke-tester chaque endpoint sans ouvrir l’UI — et si la démo live bugue, c’est mon backup.



streamlit_app.py — httpx → API_URL (:8000)

2026-08-05

UI — role dashboards

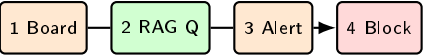
L'interface, c'est là où la règle two-channel devient visible — le login existe pour donner la bonne vue à la bonne personne.

L'auth vit dans auth.py avec des passwords hashés, et deux roles : un compte coach, et des comptes learner, student un à vingt-quatre.

En coach, on a la vue d'ensemble : le risk board, le stream d'alerts, et les metrics — tout pour décider qui aider en premier.

En learner, c'est un autre monde : l'id est verrouillé, les features peuvent être préremplies depuis le board, et les messages partent vers chat. Surtout, cette vue n'affiche que le field reply — il n'y a aucun widget capable d'afficher un coach alert. Voilà la règle two-channel, concrète là où l'utilisateur touche.





- Watch route, Day citation, channel split, guardrails
- API :8000 + UI :8501 must both run

2026-08-05

EduCoach AI

Demo agenda

Je vous dis quoi regarder, pour que les quatre beats soient des démos volontaires, pas du clic au hasard.

Beat un : je me connecte en coach et je rafraîchis un day — le Random Forest ranke toute la classe et compte qui est at-risk. Le numbers brain.

Beat deux : je passe en learner et je pose une question de théorie. Regardez le routing vers le Tutor et la réponse qui cite un day précis. C'est le retrieval qui ancre la réponse dans nos leçons.

Beat trois : j'attache des activity features faibles. Regardez le split — l'étudiant reçoit encore une réponse normale et encourageante, pendant que le coach reçoit tranquillement une alert. Un message, deux canaux.

Beat quatre : j'envoie une string de prompt-injection et les guardrails la bloquent avant le modèle. Risque, retrieval, le split, et la safety — tout le système en miniature.

Demo agenda



Live demo

Narrate implementations: RAG → graph → RF alert

2026-08-05

EduCoach AI

Live demonstration

Live demonstration

Live demo

Narrate implementations: RAG → graph → RF alert

Moment de passer au système qui tourne — l’API sur le port huit mille, l’UI sur huit cinq zéro un. En cliquant les quatre beats, je nommerai le fichier qui travaille à chaque étape, pour lier l’écran au code.

Note à moi-même si ça foire : rester calme et retomber sur la page Swagger. Je peux rejouer les mêmes quatre beats là et montrer de vrais résultats. La démo rend l’architecture tangible — ce n’est pas l’argument lui-même, donc un hiccup ne casse pas le travail.

Piece	Outcome
RF recall / RMSE	1.0 / $\approx$ 2.65
RAG grounded	$\sim$ 131 chunks, Top-3
LangGraph + channels	OK
API + role UI	OK
Pytest core paths	Green

Results checklist

Petit récap de ce qu'on a livré. Le Random Forest est en place, recall-first, donc le coach ne rate pas les strugglers. Le retrieval est grounded dans nos leçons, avec citations. L'orchestrator tourne avec les deux canaux bien séparés. FastAPI donne des contrats validés, l'UI force les deux roles, et les tests core passent.

Et un point exprès, parce que l'intégrité compte : la cible soft d'erreur a été un peu manquée, et on le dit ouvertement. C'était un trade volontaire — recall parfait sur les at-risk contre une erreur de score un peu plus haute — et pour un outil de coaching, c'est le bon call.

Piece	Outcome
RF recall / RMSE	1.0 / $\approx$ 2.65
RAG grounded	$\sim$ 131 chunks, Top-3
LangGraph + channels	OK
API + role UI	OK
Pytest core paths	Green

- Ollama freeze → pluggable get_llm() / Gemini
- Chroma version → ingest rebuild
- Thin lessons → enrich code/Output
- Supervisor “for” bug → theory-first routing
- Next: Real data mapping, real auth, durable memory

2026-08-05

EduCoach AI

└─ Difficulties & next (engineering)

Je ferme la partie technique honnêtement — une soutenance est plus forte quand on nomme ce qui a foiré.
Premier blocker : l’inference locale. Un modèle à trois milliards de paramètres via Ollama a freeze la machine. Le fix, ce n’était pas d’abandonner l’architecture, mais de rendre le provider pluggable via get_llm(), pour passer à Gemini sans toucher un agent.
Deuxième, opérationnel : un changement de version Chroma a cassé un ancien index. On a appris à traiter un rebuild comme routine, pas comme panique.
Troisième, un souci de qualité, pas un bug : les premières leçons étaient trop maigres, donc le Tutor hésitait. Les enrichir avec du vrai code et de l’output a plus fixé le retrieval que n’importe quel prompt tweak.
Quatrième, le routing : « for » ressemblait à du code, donc « What is a for loop ? » allait au Helper. Le routing theory-first a fixé ça.
Next steps : mapper un vrai dataset genre OULAD dans notre schema, vraie authentication, memory durable, et un jour un supervisor basé modèle.





Questions?

2026-08-05

EduCoach AI

Thank you

Voilà EduCoach tel qu'on l'a construit : il prédit le risque tôt pour le coach, tutorise les apprenants depuis nos propres leçons, et garde les alerts du coach hors du chat de l'étudiant — appliqué dans le state, l'API, et l'UI.
Merci — je suis prêt pour vos questions.

Thank you

EduCoach AI — implementation deep dive

Questions?